Recursion

- *A recursive definition* is one which uses the concept being defined in the definition itself.

- All recursive definitions should have a non-recursive part, called the *base case*; otherwise, the recursive call will never get terminated and will fall in infinite loop.

- We focus only finite recursion.

Recursive Definition

Contain two parts:

Base case. We must always have some base case, which can be solved without recursion.

Making progress. For the cases that are to be solved recursively, the recursive call must always make progress toward a base case.

Recursive Algorithm

An algorithm is called recursive if it solves a problem by reducing it to an instance of the same problem with smaller input.

Problem: Computing a^n; $a \neq 0$, $n>0$

Solution: $a^n = a * a^{n-1}$ for $n>0$; $a^0 = 1$

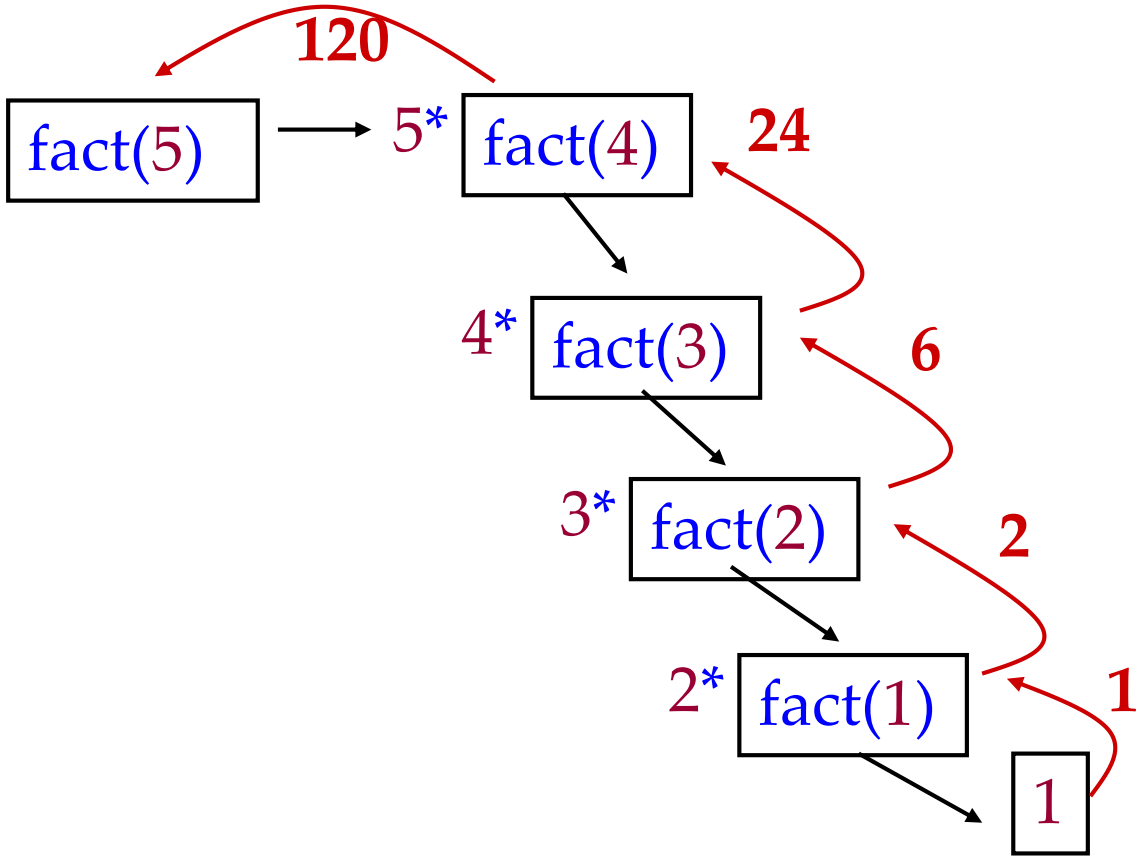
Linear Recursion

- **Base cases:**
  - Begin by testing for a set of base cases.
- **Recur once:**
  - Linear recursion occurs whenever there is **one** recursive call for each non-base case.

Example of Linear Recursion

Factorial(n)

```
fact(n)   =   if (n==1) return 1;
              else return n * fact(n-1);
```



More examples

Multiplication of two positive integers m and n is defined as

Base case. mult(1) = m

Non-base case. mult(n) = mult(n-1) + m for $n \geq 2$

More examples

A sequence T represents a list of objects in some order.

Example:

1. $T(1) = 1$
2. $T(n) = T(n-1) + 3$ for $n \geq 2$

Recursive Squaring Method

Algorithm Power(x, n):

Input: A number x and integer n

Output: The value x^n

if $n = 0$ **then**

return 1

if n is odd **then**

y = Power(x, (n - 1)/ 2)

return $x \cdot y \cdot y$

else

y = Power(x, n/ 2)

return $y \cdot y$

$$p(x,n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot p(x,(n-1)/2)^2 & \text{if } n > 0 \text{ is odd} \\ p(x,n/2)^2 & \text{if } n > 0 \text{ is even} \end{cases}$$

Complexity of the Recursive Squaring Method

Algorithm Power(x, n):
Input: A number x and integer n
Output: The value x^n

```
if n = 0 then
    return 1
if n is odd then
    y = Power(x, (n - 1)/ 2)
    return x · y · y
else
    y = Power(x, n/ 2)
    return y · y
```

Each time we make a recursive call we halve the value of n; hence, we make log n recursive calls. The method runs in $O(\log n)$. Why?

Reversing an Array

Algorithm ReverseArray(*A*, *i*, *j*):

Input: An array *A* and nonnegative integer indices *i* and *j*

Output: The reversal of the elements in *A* starting at index *i* and ending at *j*

if $i < j$ **then**

Swap $A[i]$ and $A[j]$

ReverseArray(A, $i + 1$, $j - 1$)

return

Binary Recursion

Binary recursion occurs whenever there are **two** recursive calls for each non-base case.

Example - Fibanacci Numbers

- Fibonacci numbers are defined recursively:

  $F_0 = 0$

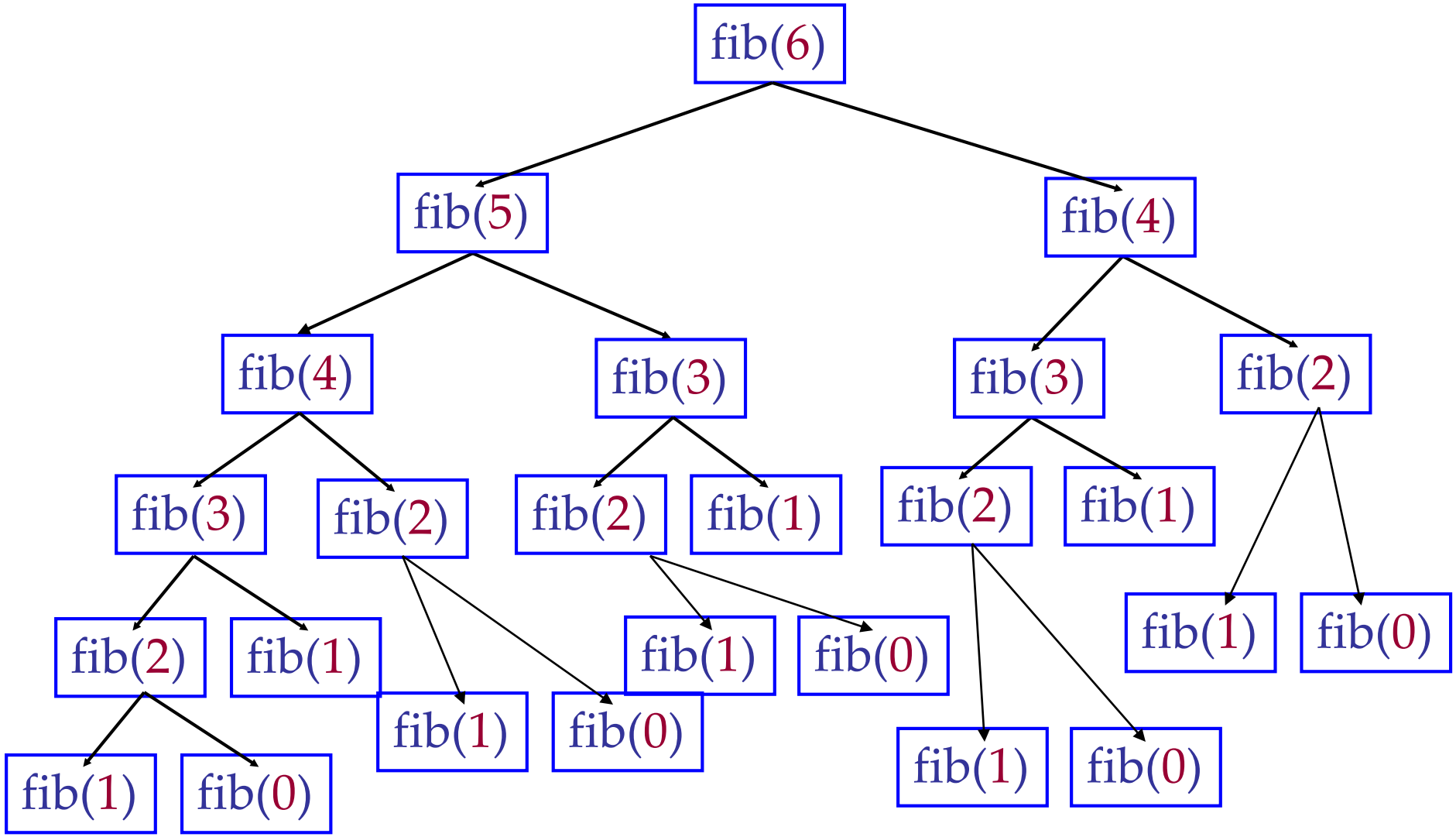
  $F_1 = 1$

  $F_i = F_{i-1} + F_{i-2}$ for $i > 1$.

- Recursive algorithm:

```
Algorithm BinaryFib(k):
  if k = 1 then
    return k
  else
    return BinaryFib(k - 1) + BinaryFib(k - 2)
```

Fibonacci Calls



Too many duplicate calls!

A Better Fibonacci Algorithm

Use of linear recursion

```
j=0, i=1;
Algorithm LinearFibonacci(k):
   if (k!=1)
            t = i;
            i = i+j;
            j = t;
            print (j);
            LinearFibonacci(k - 1)
   return;
```

An iterative version of Fibonacci series

```
int fib(int n)
{
   int fibn = 0, fibn1 = 0, fibn2 = 1;

   if (n < 2)
      return n;
   else
     {
        for( int i = 2; i ≤ n; i++ ) {
           fibn = fibn1 + fibn2;
           fibn1 = fibn2;
           fibn2 = fibn;
        }
     return fibn;
     }
}
```

Problems with Recursion

- Recursion is generally favored over iteration because it is easier to analyze formally.

- However, it can be less efficient, as more functional calls and stack operations involved in it.

Tail Recursion

Tail recursive method has the recursive call as the last statement in the method.

Tail recursion

```
void tail(int i) {
if (i>0) {
   printf("value is %d", i);
tail(i-1);
}
```

Non-tail recursion

```
int fact(int x){
   if (x==0)
      return 1;
   else
      return x*fact(x-1);
}
```

Advantage of Tail Recursive Method

- Tail Recursive methods are easy to convert to iterative.

```
void tail(int i){
  if (i>0) {
    prinf("value is %d", i);
    tail(i-1);
  }
}
```



```
void iterative(int i){
  for (;i>0;i--)
    prinft("value is %d", i);
}
```

- Smart compilers can detect tail recursion and convert it to iterative version to optimize code.

Converting Non-tail to Tail Recursive

A non-tail recursive method can be converted to a tail-recursive method by means of an "auxiliary" parameter used to form the result.

```
int fact_aux(int n, int result) {
   if (n == 1)
      return result;
   return fact_aux(n - 1, n * result)
}
int fact(n) {
   return fact_aux(n, 1);
}
```

Converting Non-tail to Tail Recursive

A tail-recursive Fibonacci method can be implemented by using two "auxiliary" parameters for accumulating results.

auxiliary parameters!

```
int fib_aux( int n , int next, int result)
{
  if (n == 0)
    return result;
  return fib_aux(n - 1, next + result, next);
}

int fib(n) {
   return fib_aux(n, 1, 0);
}
```